

KOGI

Independent Worker Operating System

Portfolio Management System — The Master Spreadsheet

kogi-portfolio · KPMS · KPSS · KPRG · v3.0 · March 2026

System Design Document

Root Portfolio Data Substrate · Spreadsheet Architecture · Component Registry · Derived Sheets · Column System · View Engine · Computation Models · Engine Integrations · API · Persistence

Module / Subsystem	Description
KPMS — Portfolio Management System	The master orchestration layer: spreadsheet substrate, row/column model, sheet registry, view engine
KPSS — Portfolio Spreadsheet Substrate	The low-level data engine: PortfolioRow, ColumnSchema, SheetDefinition, indexed store
KPRG — Portfolio Component Registry	Canonical registry of all PortfolioComponents system-wide; the master index
KPVW — View Engine	Filter · Sort · Group · Pivot · Slice across any column dimension
KPCM — Computation Engine	11+ analytical models; derived column computation; rollup and aggregation
KPCL — Collaborative Editing	CRDT-backed concurrent multi-user spreadsheet editing
KPEX — Export & Integration	CSV · XLSX · JSON · API · Embed · Webhook export of any sheet or view

1. Overview & Vision

Design Principle: Every entity in the Kogi ecosystem — programs, projects, resources, assets, artifacts, gigs, benefits, finances, relationships, governance actions, analytics events — is a row in one universal, scalable, configurable spreadsheet. The Portfolio Management System is that spreadsheet: the master, living, computed record of an independent worker's entire operational and creative world.

The Kogi Portfolio Management System (kogi-portfolio, KPMS) is the central, sovereign data substrate of the entire platform. It is architected as a large-scale, configurable, multi-dimensional spreadsheet: a universal record of everything the independent worker owns, operates, creates, manages, finances, and shares.

Unlike a conventional spreadsheet tool, the Portfolio Spreadsheet System (KPSS) is a living, event-sourced, AI-augmented data structure. Every row is a PortfolioComponent with full lifecycle management, CRDT-based distributed consistency, policy-governed access control, and a complete append-only EventLog. Every column is a typed, indexed, computable field. Every sheet is a materialized, filtered, aggregated view over the single underlying component registry.

The system is simultaneously the foundational data model from which every kogi application reads, the operational dashboard from which workers manage their entire portfolio of work and life, and the intelligence substrate from which kogi-engine extracts signals, computes recommendations, and surfaces AI-driven insights.

1.1 Core Metaphor: The Portfolio as Master Spreadsheet

A portfolio is not a folder or a gallery — it is a master spreadsheet. Rows are portfolio components. Columns are the properties, metrics, relationships, and computed values attached to those components. Sheets are curated, filtered, and aggregated views of the row set, organized around different dimensions of the worker's world: projects, finances, resources, benefits, gigs, collaborators, and more.

Spreadsheet Concept	Portfolio System Mapping
Workbook	PortfolioSystem — the root container holding all sheets for a worker or organization
Sheet	A named, scoped, filterable materialized view of PortfolioComponents (e.g., Projects Sheet, Finances Sheet, Benefits Sheet)
Row	A PortfolioComponent — any item, container, resource, artifact, asset, or derived entity
Column	A typed PortfolioColumn — a field, computed metric, relationship reference, or engine-derived signal on a component
Cell	The intersection of a PortfolioRow and a PortfolioColumn — a single typed, versioned, audited value
Formula	A ComputedColumn — a derived field calculated from other columns via the Computation Engine (e.g., Budget Remaining = Budget Allocated – Budget Spent)
Filter	A ViewFilter — a boolean predicate applied to rows (e.g., Status = Active, Type = Project, Owner = me)
Sort	A ViewSort — ordering applied to rows (e.g., sort by Health Score descending, then by Created Date ascending)
Group	A ViewGroup — hierarchical aggregation of rows by a shared column value (e.g., group by Program, group by Status)

Pivot Table	A PivotView — cross-dimensional aggregation (e.g., Budget Spent by Owner × Program × Quarter)
Chart	A ChartView — visual rendering of a column's values (bar, line, gauge, treemap, network graph)
Freeze Panes	PinnedColumns — identity and key status columns always visible during horizontal scroll
Conditional Formatting	RowHighlightRule — color-coding rows based on column values (e.g., red for overdue, green for on-track)
Import / Export	KPEX — CSV, XLSX, JSON-LD export of any sheet; webhook push; kogi-dev API pull

1.2 Design Goals

- **Universal Abstraction** — Every platform entity — from a single task to a federation portfolio — is representable as a row in this system.
- **Infinitely Configurable** — Columns are addable, reorderable, hideable, pinnable, and computable. Sheets are addable, filterable, and shareable. Views are user-defined, template-driven, and AI-suggested.
- **Event-Sourced & Auditable** — Every cell mutation is appended to the component's EventLog. No data is ever deleted — only archived or superseded.
- **Distributed-First** — VectorClock + CRDT log (LWW + OR-Set) ensures that concurrent edits across nodes, collaborators, and federated portfolios converge correctly.
- **AI-Augmented** — Computed columns, smart defaults, anomaly flags, health scores, risk scores, match scores, and proactive suggestions are generated by kogi-engine and surfaced directly in the spreadsheet.
- **Policy-Governed** — Every row, column, and cell respects a tiered PermissionTier system. Row-level and column-level access control allows fine-grained sharing.
- **Composable & Extensible** — The PortfolioPlugin trait and ToolBox integration allow developers and power users to extend the system with custom column types, computed models, and automated row generators.
- **Collaborative** — Multi-user real-time collaborative editing via CRDT; contribution attribution via ContributionRecord; shared portfolio governance via GovernanceProposal.

2. System Architecture

2.1 Structural Overview

The Portfolio Management System is organized in five distinct layers, from the raw data substrate up to the user-facing view and intelligence layers:

Layer	Components & Responsibility
L0 — Substrate	PortfolioComponent (Rust core) · ComponentMetadata · ComponentData · EventLog · CrdtLog · VectorClock · GraphEdge · PolicyEngine
L1 — Registry	KPRG Master Component Registry · ComponentIndex · OwnerIndex · TagIndex · TypeIndex · StatusIndex · FullTextIndex (SearchEngine)
L2 — Spreadsheet Engine	KPSS PortfolioRow · ColumnSchema · SheetDefinition · CellStore · ColumnComputer · Aggregation Engine
L3 — View Engine	KPVW ViewFilter · ViewSort · ViewGroup · PivotView · ChartView · RowHighlightRule · PinnedColumns · ViewTemplate
L4 — Intelligence Layer	kogi-engine ComputedColumns · HealthScoreColumn · RiskScoreColumn · MatchScoreColumn · RecommendationColumn · AnomalyFlagColumn · Oba AI overlay
L5 — Application Layer	kogi-portfolio UI · kogi-office Boards · kogi-bank Finance Views · kogi-marketplace Asset Views · kogi-community Feed Integration · KPEX Export

2.2 Component Taxonomy (Row Types)

The complete taxonomy of PortfolioComponents — the rows that populate the master spreadsheet:

Category	Row Type	Description
item:portfolio	Portfolio	A named top-level portfolio representing the worker's entire work domain, a specific practice area, or a cooperative's shared work.
item:portfolio	SubPortfolio	A scoped sub-portfolio nested within a parent Portfolio — a focus area, division, or thematic cluster.
item:program	Program	A strategic initiative grouping multiple related Projects toward a shared objective. Carries budget, timeline, and KPI tracking.
item:project	Project	A bounded, deliverable-producing unit of work with methodology (Agile, Kanban, Waterfall, custom), sprints, backlog, and releases.
item:resource	Resource	A human, financial, equipment, or service resource that can be allocated to Projects, Programs, or Gigs.
item:artifact	Artifact	A typed output document, file, design, code, or generated asset produced by a Project or Program. Carries versioning and provenance.
item:asset	Asset	A capital or intellectual asset — digital, physical, financial, or IP — owned or managed by the worker or organization.
item:benefit	BenefitAccount	A portable benefit account (HSA, retirement, PTO, professional development) — full lifecycle and analytics as a portfolio item.
item:gig	Gig	A single gig engagement: ride, delivery, task, session, or booking — income-generating event mapped to portfolio.

item:contract	Contract	A formal agreement with a client, cooperative, or employer. Carries payment terms, deliverables, and expiry.
item:job	Job	A longer-term employment or consulting engagement. Carries rate, hours, organization ref, and benefits linkage.
item:task	Task	A single atomic unit of work within a Project or Program backlog.
item:campaign	Campaign	An equity crowdfunding, grants, or microfinancing campaign — tracked as a portfolio item with target, raised, investor list.
item:grant	Grant	A grant application or awarded grant — tracks application status, amounts, milestones, and reporting obligations.
item:investment	Investment	An equity stake, revenue-share instrument, SAFE note, or pool investment — tracked with valuation and cap table position.
item:profile	Profile	A user or organization profile (see KPRF) — surfaced as a row in community and directory sheets.
container:binder	Binder	A logical collection of items organized by shared purpose, client, topic, or workflow — unordered, flexible.
container:book	Book	A structured, navigable document-like container. Subtypes: Notebook, ContactBook, PlayBook, ScheduleBook, PlanBook, GuideBook, ItemBook.
container:record	Record	A formal, auditable record container — equivalent to a file folder in a records management system.
container:folder	Folder	A general-purpose hierarchical container for organizing items and other containers.
container:registry	Registry	A structured, searchable catalog of items — e.g., a skills registry, asset registry, or vendor registry.
container:archive	Archive	Deep-storage container for completed, retired, or historical items. Full restore capability.

2.3 GraphEdge Taxonomy (Relationship Columns)

Rows in the master spreadsheet are connected by typed, directional GraphEdges. Relationships are first-class data — they appear as computed column groups in any sheet:

Edge Type	Direction	Semantics
Hierarchy	parent → child	Structural containment: Portfolio contains Program; Program contains Project; Project contains Task
Dependency	blocker → blocked	Work dependency: Task B cannot start until Task A is complete
Link	sibling ↔ sibling	Loose association between any two components — cross-reference without structural implication
Contains	container → item	A Container holds Items (Binder contains Artifacts; Book contains Records)
Federation	portfolio ↔ portfolio	Cross-system CRDT sync between federated portfolio nodes
ResourceShare	resource → entity	A resource is shared with a Space, Org, or User at a specified access level
Attribution	contributor → component	A ContributionRecord linking a contributor entity to a shared portfolio component
Investment	investor → campaign	An equity or revenue-share investment linking an investor profile to a campaign item

Derives	source → derived	A derived item (e.g., a sub-artifact) that was produced from a source component
Governs	policy → component	A governance policy applied to a component
BelongsTo	item → organization	Membership of an item in an organization, collective, or cooperative

3. Universal Row Schema (PortfolioRow)

Every row in the master spreadsheet implements the PortfolioRow schema — a superset of ComponentMetadata + ComponentData + all derived and computed fields. Rows are typed: the active columns for a given row are determined by its ItemType or ContainerType, but the underlying storage schema is universal so that cross-type aggregation, filtering, and sorting works uniformly across all sheets.

3.1 Column Groups Overview

Column Group	Columns (count)
Identity	8
Taxonomy	7
Lifecycle	8
Ownership & Users	12
Relationships	10
Version Control	5
Timeline & Schedule	12
Financial & Budget	14
Resource Allocation	8
Work & Delivery	10
Governance & Policy	9
Analytics & Metrics	18
Visibility & Access	5
Tags & Discovery	6
AI & Engine Signals	9
Collaboration	8
Benefits (BenefitAccount rows)	10
Gig & Contract (Gig/Contract rows)	10

3.2 Identity Column Group

Column	Type	Description
component_id	UUID	Globally unique immutable identifier for this row
slug	String	URL-safe short identifier (unique within owner scope)
component_type	ComponentType enum	Item Container — top-level type discriminant
item_type	Option<ItemType>	Portfolio Program Project Resource Artifact Asset BenefitAccount Gig Contract Job Task Campaign Grant Investment Profile

container_type	Option<ContainerType>	Binder Book Record Folder Registry Archive
name	String	Primary display name of the component
display_name	Option<String>	Override display name (e.g., short title for board cards)
icon	Option<String>	Icon reference (emoji, icon set key, or custom URL)
color	Option<HexColor>	Color tag for board and calendar views

3.3 Lifecycle Column Group

Column	Type	Description
status	ComponentStatus	Draft Active Paused Completed Archived Cancelled Suspended
state	ComponentState	Idle InProgress Blocked UnderReview Approved Published Merged Locked
visibility	Visibility	Private Protected Internal Public Custom
lifecycle_stage	LifecycleStage	Concept Planning Execution Review Delivery Complete Post-Mortem
created_at	DateTime<Utc>	Row creation timestamp
updated_at	DateTime<Utc>	Last mutation timestamp
archived_at	Option<DateTime<Utc>>	Archive timestamp (set on transition to Archived status)
deleted_at	Option<DateTime<Utc>>	Soft-delete timestamp

3.4 Financial & Budget Column Group

Column	Type	Description
budget_allocated	Option<Decimal>	Total budget allocated to this component
budget_spent	Decimal	Cumulative spend recorded via record_spend()
budget_remaining	ComputedColumn	= budget_allocated – budget_spent (engine-computed)
budget_utilization_pct	ComputedColumn	= (budget_spent / budget_allocated) × 100
currency	CurrencyCode	ISO 4217 currency code (e.g., USD, EUR, GBP)
rate	Option<Decimal>	Hourly, daily, project, or gig rate
rate_type	RateType	Hourly Daily Weekly Monthly ProjectFixed PerUnit Revenue%
revenue	Decimal	Cumulative revenue generated by or attributed to this component
expenses	Decimal	Cumulative expenses charged against this component
profit	ComputedColumn	= revenue – expenses
roi	ComputedColumn	= (revenue – budget_spent) / budget_spent × 100
tax_category	TaxCategory	Business SelfEmployment PassThrough Exempt Investment Benefit
payment_terms	Option<String>	Net-30, Net-60, milestone, upfront, subscription, etc.
invoice_refs	Vec<InvoiceId>	References to kogi-bank invoices linked to this component

bank_account_ref	Option<AccountId>	kogi-bank account receiving or funding this component's transactions
------------------	-------------------	--

3.5 Timeline & Schedule Column Group

Column	Type	Description
start_date	Option<Date>	Planned or actual start date
end_date	Option<Date>	Planned or actual end date
due_date	Option<Date>	Hard deadline for delivery or completion
estimated_duration_days	Option<u32>	Estimated total duration in calendar days
actual_duration_days	ComputedColumn	= (end_date or today) – start_date in days
progress_pct	f32	Completion percentage (0–100); auto-computed from child task completion if type=Project
schedule_variance_days	ComputedColumn	= actual_duration_days – estimated_duration_days; negative = ahead of schedule
milestones	Vec<Milestone>	Named, dated milestone events within this component's timeline
sprint_refs	Vec<SprintId>	References to sprints within this project
program_increment	Option<PI>	SAFe-style Program Increment reference (quarter-length planning window)
quarter	Option<String>	Fiscal or calendar quarter label (e.g., Q1-2026)
fiscal_year	Option<u32>	Fiscal year for budget and reporting alignment

3.6 Governance & Policy Column Group

Column	Type	Description
policy_ids	Vec<PolicyId>	Active governance policies attached to this component
governance_model	GovernanceModel	None OwnerDecision AdminApproval MajorityVote SupermajorityVote Consensus MultiSig
approval_status	ApprovalStatus	NotRequired Pending Approved Rejected Escalated
approval_queue	Vec<ApprovalRequest>	Outstanding approval requests with approver, deadline, and context
compliance_flags	Vec<ComplianceFlag>	Regulatory or platform compliance issues flagged on this component
risk_flags	Vec<RiskFlag>	Risk items: severity, probability, mitigation owner, resolution status
last_reviewed_at	Option<DateTime<Utc>>	Timestamp of last governance or compliance review
charter_ref	Option<ComponentId>	Reference to the Charter artifact that governs this item
regulatory_tags	Vec<String>	Regulatory regime tags: RegCF ERISA HIPAA SOC2 GDPR custom

3.7 Analytics & Metrics Column Group

Column	Type	Description
--------	------	-------------

health_score	ComputedColumn	PortfolioHealth model output (0–100): composite of progress, budget, risk, engagement, resource adequacy
risk_score	ComputedColumn	RiskEngine output (0–100): probability-weighted aggregate of risk_flags
match_score	ComputedColumn	MatchEngine score for this row in the context of a search or opportunity query
engagement_rate	ComputedColumn	$\text{= total_engagements / reach} \times 100$ (follower-normalized engagement)
views	u64	Cumulative view count
clicks	u64	Cumulative click-through count
ctr	ComputedColumn	$\text{= clicks / views} \times 100$
shares	u64	Share and repost count
followers	u64	Current follower/subscriber count for this component
kpi_refs	Vec<KPI>	Key Performance Indicators: name, target, current, unit, trend
okr_refs	Vec<OKR>	Objectives and Key Results linked to this component
velocity	ComputedColumn	Story points or tasks completed per sprint (Project rows only)
throughput	ComputedColumn	Items completed per time period (kanban-style metric)
cycle_time_days	ComputedColumn	Avg days from In-Progress to Completed for child tasks
lead_time_days	ComputedColumn	Avg days from Created to Completed for child tasks
collaboration_score	ComputedColumn	CollaborationEngine output: contributor diversity $\times$ velocity $\times$ governance participation
sentiment_score	ComputedColumn	Sentiment analysis of comments, reviews, and community mentions
anomaly_flags	Vec<AnomalyFlag>	Engine-detected anomalies: budget spike, velocity drop, engagement cliff, risk escalation

4. Sheet System — Derived Sheet Taxonomy

A Sheet is a named, scoped, materialized view of the PortfolioComponent registry. Every sheet is defined by: a SheetDefinition (filter predicates, visible columns, default sort, default grouping), a ColumnSchema (the ordered set of columns visible on this sheet), and an optional ComputedColumnSet (additional engine-derived columns rendered only on this sheet). Sheets are not separate databases — they are views over the single underlying PortfolioRow store.

4.1 Master Sheet Registry

Sheet ID	Sheet Name	Primary Row Types	Default Group By
SHT-001	Master Registry	All types	Type then Status
SHT-002	Portfolio Hierarchy	Portfolio, SubPortfolio, Program, Project	Hierarchy (tree)
SHT-003	Programs	Program	Status
SHT-004	Projects	Project	Program → Status
SHT-005	Tasks & Backlog	Task, Story, Epic, Feature	Project → Sprint
SHT-006	Resources	Resource	Resource Type
SHT-007	Assets	Asset	Asset Type
SHT-008	Artifacts	Artifact	Project → Artifact Type
SHT-009	Finances	All financial rows	Quarter → Category
SHT-010	Budget Tracker	Program, Project, Gig, Contract	Program → Status
SHT-011	Work & Gigs	Gig, Contract, Job, Task	Platform → Status
SHT-012	Deliverables	Project, Artifact, Release	Project → Status
SHT-013	Timeline	All dated rows	Quarter → Program
SHT-014	Roadmap	Program, Project, Milestone	Program → Quarter
SHT-015	Portable Benefits	BenefitAccount	Benefit Type
SHT-016	Grants & Microfinancing	Grant, Campaign	Status
SHT-017	Equity & Crowdfunding	Campaign, Investment	Campaign Type
SHT-018	Group Economics	Shared Portfolio, Revenue Pool, Org	Organization
SHT-019	Organizations	Profile (org type), Binder (org)	Org Type
SHT-020	Shared Portfolios	Portfolio (shared)	Owner Org
SHT-021	Collaboration	All shared/contributed	Contributor
SHT-022	Risk Register	All rows with risk_flags	Risk Severity
SHT-023	Analytics Dashboard	All rows	Type → Health Score
SHT-024	Feed & Community	Profile, Post, Artifact (published)	Space
SHT-025	Contacts & Network	Profile, Contact	Relationship Type
SHT-026	Marketplace Listings	Asset, Resource, Gig (public)	Category
SHT-027	Exchange	Investment, Deal, Campaign, Asset	Exchange Type

SHT-028	Archive	All archived rows	Archive Date
SHT-029	Event Log	EventLog entries (read-only)	Date → Actor
SHT-030	Custom Sheet	User-defined	User-defined

5. Sheet Detailed Designs

5.1 Master Registry Sheet (SHT-001)

The Master Registry is the root spreadsheet — the unfiltered, full-column view of every PortfolioComponent the authenticated user can access. It is the entry point for global search, bulk operations, cross-type analytics, and administrative management. Every other sheet is a subset or derived view of the Master Registry.

Pinned Columns (always visible)	component_id · slug · component_type · item_type · name · status · visibility · owners
Default Visible Columns	+ created_at · updated_at · progress_pct · health_score · risk_score · due_date · budget_remaining · tags
Optionally Visible	All remaining columns — user-toggable per session or saved view
Default Sort	updated_at descending
Default Group	component_type
Row Actions	Open · Edit · Duplicate · Archive · Share · Export · Add to Sheet · Pin · Set Status
Bulk Actions	Set Status · Set Visibility · Assign Tags · Archive · Export Selected · Add to Binder
AI Overlay	Oba surfaces anomalies, stale items (not updated > 30 days), health warnings, and quick-action suggestions inline

Illustrative Master Registry rows (showing 8 columns of the universal row):

ID (short)	Name	Type	Status	Owner	Health	Budget Rem.	Due Date
PRJ-0041	API Gateway Refactor	Project	Active	@alex	84	\$4,200	Apr 15
PRG-0007	Platform Q2 Release	Program	Active	@alex	71	\$18,500	Jun 30
GIG-1203	DoorDash Delivery Block	Gig	Completed	@alex	—	—	Mar 12
BEN-0003	HSA Account 2026	BenefitAccount	Active	@alex	—	\$2,840	Dec 31
ART-0088	API Spec v2 (OpenAPI)	Artifact	Draft	@alex	—	—	Apr 1
AST-0012	Podcast Equipment Kit	Asset	Active	@alex	—	\$0	—
GRT-0005	SBIR Phase I Application	Grant	Submitted	@alex	—	\$50,000	May 15
CPG-0002	Studio Collective Fund	Campaign	Active	@org	67	\$12,000	Jul 1

5.2 Portfolio Hierarchy Sheet (SHT-002)

The Portfolio Hierarchy Sheet renders the structural tree of the portfolio — Portfolio → SubPortfolio → Program → Project → Task — as an expandable, indented spreadsheet. Each level is indented by its depth in the

GraphEdge::Hierarchy tree. Rollup columns aggregate child metrics upward: budget, progress, health score, and risk.

Column	Type / Computation	Notes
depth_indicator	Computed indentation	Visual indent rendering tree level (0 = Portfolio, 1 = SubPortfolio/Program, 2 = Project, 3 = Task)
name	String	Indented display name
component_type	ComponentType	Type label with icon
status	ComponentStatus	Lifecycle status
child_count	ComputedColumn	Count of direct child rows in the hierarchy
total_descendant_count	ComputedColumn	Count of all transitive children (deep tree size)
progress_pct (rollup)	ComputedColumn	Weighted average of child progress percentages
budget_allocated (rollup)	ComputedColumn	Sum of all descendant budget allocations
budget_spent (rollup)	ComputedColumn	Sum of all descendant actual spend
health_score (rollup)	ComputedColumn	SubPortfolioRollup model: min child health weighted by budget
risk_score (rollup)	ComputedColumn	Max child risk score, severity-weighted
owners	Vec<EntityRef>	All owners at this hierarchy level
due_date	Option<Date>	Outermost deadline of this branch
expand_collapse	UI control	Expand/collapse the subtree under this row

5.3 Projects Sheet (SHT-004)

The Projects Sheet is the primary operational view for project-level work. It exposes the full ProjectPayload — methodology, sprint data, backlog size, velocity, releases — alongside financial, governance, and AI columns. Boards (Kanban, Gantt, Calendar, Agile) are alternative view modes for the same underlying rows.

Column Group	Columns	Notes
Identity	component_id · name · slug · color · icon	Standard identity group
Classification	project_type · methodology · domain · tags	ProjectType: Software Design Content Research Operations Event Custom
Lifecycle	status · state · lifecycle_stage · visibility	Full lifecycle state machine
Ownership	owners · editors · watchers	User role columns
Timeline	start_date · end_date · due_date · progress_pct · schedule_variance_days	Key scheduling columns
Sprints	sprint_count · current_sprint · sprint_velocity · backlog_size	Derived from ProjectPayload.sprints
Releases	release_count · latest_release · next_release_date	Derived from ProjectPayload.releases
Finance	budget_allocated · budget_spent · budget_remaining · budget_utilization_pct	Financial tracking

Governance	governance_model · approval_status · risk_flags · compliance_flags	Governance state
Analytics	health_score · risk_score · cycle_time_days · lead_time_days · velocity · throughput	AI-computed performance signals
Collaboration	contributor_count · collaboration_score · pending_review_count	For shared projects
AI Signals	predicted_completion_date · schedule_risk_flag · budget_risk_flag · Oba_hint	Engine-generated forward-looking signals
ToolBoxes	toolbox_ids · toolbox_names	Attached TMS ToolBoxes

5.4 Tasks & Backlog Sheet (SHT-005)

Tasks represent the atomic unit of work. This sheet aggregates all Tasks, Stories, Epics, Features, and work items across all projects — queryable globally or scoped to a project, sprint, or program. It supports drag-and-drop priority ordering, inline editing, and bulk status transitions.

Column	Type	Description
name	String	Task name / story title
story_type	StoryType	Feature Bug Testing Capability Issue Defect Enhancement Enabler Blocker UseCase Requirement Documentation Milestone Risk Deployment Archive
parent_project	ComponentRef	Project this task belongs to
epic_ref	Option<ComponentRef>	Epic this story belongs to
sprint_ref	Option<SprintId>	Sprint this task is committed to
status	ComponentStatus	Draft Active InProgress InReview Blocked Completed Cancelled
priority	Priority	Critical High Medium Low Backlog
story_points	Option<u32>	Estimation in story points
assigned_to	Vec<UserId>	Assigned user(s)
due_date	Option<Date>	Task deadline
labels	Vec<String>	Categorization labels
dependencies	Vec<ComponentId>	Tasks that must complete before this task can start
dependents	Vec<ComponentId>	Tasks blocked by this task
acceptance_criteria	Vec<String>	Definition of Done criteria
attachments	Vec<FileRef>	Linked files, artifacts, and documents
time_tracked	Option<Duration>	Actual time logged against this task
created_at / updated_at	DateTime<Utc>	Timestamps

5.5 Finances Sheet (SHT-009)

The Finances Sheet is a unified financial ledger view spanning all portfolio components that carry financial data. It aggregates income, expenses, budget, and asset values across gigs, contracts, projects, campaigns, benefits,

and investments. It is the master financial spreadsheet — the equivalent of a real-time P&L across the worker's entire portfolio of work.

Column	Type	Applies To	Notes
name	String	All	Component display name
component_type	ComponentType	All	Row type for filtering
quarter	String	All	Fiscal quarter label for period grouping
fiscal_year	u32	All	Fiscal year
revenue	Decimal	Gig, Contract, Job, Campaign, Investment	Income or revenue generated
expenses	Decimal	Project, Program, Asset, Gig	Costs incurred
budget_allocated	Decimal	Program, Project	Planned budget
budget_spent	Decimal	Program, Project	Actual spend
budget_remaining	ComputedColumn	Program, Project	= allocated – spent
profit	ComputedColumn	All	= revenue – expenses
roi	ComputedColumn	Asset, Investment, Campaign	Return on investment %
tax_category	TaxCategory	All	For tax report generation
invoice_refs	Vec<InvoiceId>	Contract, Gig, Job	Linked kogi-bank invoices
payment_status	PaymentStatus	Contract, Gig, Job	Unpaid Partial Paid Overdue Disputed
gig_earnings	Decimal	Gig rows	Base earnings before tips and deductions
tips	Decimal	Gig rows	Tip income
portable_contribution	Decimal	Gig rows	Platform contribution to portable savings (e.g. 4% of pre-tip)
benefit_contributions	Decimal	BenefitAccount rows	Cumulative contributions to benefit account
benefit_balance	Decimal	BenefitAccount rows	Current account balance
investment_value	Decimal	Investment, Asset rows	Current estimated valuation
distributions_received	Decimal	Investment rows	Cumulative distributions received
grant_awarded	Decimal	Grant rows	Total grant award amount
grant_disbursed	Decimal	Grant rows	Amount disbursed to date
campaign_target	Decimal	Campaign rows	Fundraising target amount
campaign_raised	Decimal	Campaign rows	Amount raised to date
campaign_pct	ComputedColumn	Campaign rows	= raised / target × 100
bank_account_ref	AccountId	All	Linked kogi-bank account

5.6 Portable Benefits Sheet (SHT-015)

Every portable benefit account is a `PortfolioItem` (kind: `BenefitAccount`) and surfaces as a row on this sheet. The Benefits Sheet is the worker's complete, real-time benefits dashboard — balances, contributions, coverage gaps, and tax optimization signals all in one place.

Column	Type	Benefit Types	Notes
name	String	All	Account display name
benefit_type	BenefitType	All	Health Dental Vision HSA Retirement PTO Disability ProfessionalDev EmergencySavings PortableSavings
provider	String	All	Benefit provider name (e.g., Stride LLC, Fidelity, Cigna)
status	ComponentStatus	All	Active Inactive Suspended Archived
balance	Decimal	HSA, Retirement, EmergencySavings, PortableSavings, ProfessionalDev	Current account balance
ytd_contributions	Decimal	HSA, Retirement, PortableSavings	Year-to-date contributions
contribution_sources	Vec<ContributionSource>	PortableSavings	Gig platforms contributing (e.g. DoorDash 4%, Lyft 3%)
platform_contribution_ytd	Decimal	PortableSavings	Total gig platform contributions YTD
vesting_pct	ComputedColumn	Retirement	Vesting schedule completion percentage
projected_balance_1yr	ComputedColumn	Retirement, PortableSavings	Engine-projected balance at 1 year given current contribution rate
coverage_gap_flag	AnomalyFlag	All	BenefitsEngine flag: coverage gap detected
pto_accrued_days	Decimal	PTO	Current accrued paid time off (days)
pto_used_ytd_days	Decimal	PTO	PTO used year-to-date
pto_remaining_days	ComputedColumn	PTO	= accrued – used
income_replacement_coverage	ComputedColumn	Disability, PTO	Coverage days vs. income gap risk
tax_treatment	TaxCategory	All	PreTax PostTax TaxFree
tax_savings_ytd	ComputedColumn	HSA, Retirement	Estimated tax savings from pre-tax contributions YTD
claim_count	u32	Health, Disability	Number of claims filed
linked_bank_account	AccountId	All	kogi-bank account linked to this benefit

5.7 Work & Gigs Sheet (SHT-011)

The Work & Gigs Sheet aggregates all income-generating engagements: individual gig sessions, ongoing contracts, freelance jobs, consulting engagements, and booking events. It is the primary earnings dashboard for the independent worker — tracking income across all platforms and engagement types in a single view.

Column	Type	Row Types	Description
name	String	All	Engagement name / gig description

engagement_type	EngagementType	All	Gig Contract Job Consulting Freelance Booking Commission
platform	String	Gig	Platform where gig was performed (DoorDash, Uber, Lyft, etc.)
client_ref	Option<ProfileRef>	Contract, Job	Client or employer profile reference
status	ComponentStatus	All	Draft Active InProgress Completed Cancelled Disputed
start_date	Date	All	Engagement start
end_date	Option<Date>	Contract, Job	Engagement end
rate	Decimal	All	Hourly, per-gig, or project rate
rate_type	RateType	All	Hourly PerGig Daily ProjectFixed Revenue%
hours_worked	Decimal	All	Total hours logged
earnings_gross	Decimal	All	Gross earnings before deductions
tips	Decimal	Gig	Tips received
expenses	Decimal	All	Direct expenses incurred for this engagement
earnings_net	ComputedColumn	All	= earnings_gross – expenses
portable_contribution	Decimal	Gig	Platform contribution to portable savings
portable_contribution_pct	Decimal	Gig	Contribution rate (e.g. 4%)
mileage	Option<Decimal>	Gig	Miles driven (for mileage deduction)
mileage_deduction	ComputedColumn	Gig	= mileage × IRS rate
payment_status	PaymentStatus	All	Unpaid Partial Paid Overdue Disputed
invoice_ref	Option<InvoiceId>	Contract, Job	Linked kogi-bank invoice
contract_status	ContractStatus	Contract	Negotiating Signed Active Delivered Closed
deliverables	Vec<ComponentRef>	Contract	Linked artifact/deliverable rows

5.8 Grants & Microfinancing Sheet (SHT-016)

This sheet consolidates all grant applications, microloan records, and community lending pool participation into a single funding pipeline view — from initial match to final disbursement and impact reporting.

Column	Description	Notes
name	Grant or loan name	Display name
funding_type	FundingType: Grant Microloan CommunityLoan MatchedFund PlatformGrant	Category discriminant
source	Funder or lender entity name	Government, foundation, platform, or community lender
status	ApplicationStatus: Researching Drafting Submitted Pending Awarded Rejected Disbursing Reporting Closed	Full application lifecycle
target_amount	Decimal	Requested or loan amount

awarded_amount	Decimal	Confirmed award amount (set on status=Awarded)
disbursed_amount	Decimal	Amount disbursed to grant bank account to date
remaining_to_disburse	ComputedColumn	= awarded – disbursed
deadline	Date	Application submission deadline
award_date	Option<Date>	Award notification date
term_months	Option<u32>	For microloans: repayment term in months
interest_rate	Option<Decimal>	For microloans: annual interest rate %
repayment_schedule	Option<RepaymentSchedule>	Monthly installments with amounts and due dates
collateral_ref	Option<ComponentId>	Portfolio asset pledged as collateral (if any)
linked_project	Option<ComponentRef>	Project this grant is funding
milestone_refs	Vec<MilestoneId>	Grant reporting milestones
impact_report_due	Option<Date>	Next impact report due date
portfolio_match_score	ComputedColumn	GrantEngine eligibility and match score
bank_account_ref	AccountId	kogi-bank Grant Account for disbursements

5.9 Equity Crowdfunding & Investment Sheet (SHT-017)

Column	Description	Notes
name	Campaign or investment name	Display name
campaign_type	CampaignType: EquityCF RevenueShare SAFE Convertible Donation Reward	Financial instrument type
status	CampaignStatus: Draft Open Funding Closed Succeeded Failed Distributing	Lifecycle status
target_amount	Decimal	Fundraising target
raised_amount	Decimal	Total committed to date
funding_pct	ComputedColumn	= raised / target × 100
investor_count	u32	Number of distinct investors
min_investment	Decimal	Minimum investment amount (Reg CF compliance)
max_raise	Decimal	Reg CF or Reg A+ raise cap
equity_offered_pct	Decimal	For equity campaigns: percentage offered
revenue_share_pct	Option<Decimal>	For revenue share: % of revenue shared
cap_table_ref	Option<CapTableId>	kogi-bank equity cap table reference
escrow_account_ref	AccountId	kogi-bank Campaign/Escrow Account
escrow_release_condition	String	Success condition for escrow release

close_date	Date	Campaign close date
regulatory_tags	Vec<String>	RegCF RegA+ SAFE accredited-only
accredited_only	Bool	Whether restricted to accredited investors
distribution_schedule	Option<DistributionSchedule>	Revenue share distribution frequency and formula
ytd_distributions	Decimal	Total distributions paid YTD
crowdfunding_engine_score	ComputedColumn	CrowdfundingEngine: campaign health, investor match quality, success probability
linked_portfolio	ComponentRef	Portfolio or program this campaign funds

5.10 Shared Portfolios & Collaboration Sheet (SHT-020 / SHT-021)

The Shared Portfolio Sheet surfaces all portfolio components whose owner set includes more than one entity — co-owned projects, cooperative portfolios, crowdresourced artifacts, and federated programs. The Collaboration Sheet provides a contribution-centric view of the same data.

Column	Description	Notes
name	Component name	Display name
portfolio_type	SharedPortfolioType: Team Organization Collective Cooperative Federation Crowdresourced	Shared type discriminant
owner_count	ComputedColumn	Count of distinct owner entities
owners	Vec<EntityRef>	All co-owners: users, orgs, collectives
contributor_count	ComputedColumn	Total unique contributors
contribution_types_open	Vec<ContributionType>	What contribution types are currently accepted: Labor Capital Asset Knowledge Artifact Code Design Data
pending_review_count	ComputedColumn	Contributions awaiting steward review or governance vote
crowdresourcing_campaign_ref	Option<CampaignId>	Active crowdresourcing campaign
review_policy	ReviewPolicy	AutoAccept StewardReview GovernanceVote
collaboration_score	ComputedColumn	CollaborationEngine output: contributor diversity × velocity × governance participation
attribution_weights	Map<EntityId,f64>	Current attribution weights per contributor
last_contribution_date	DateTime<Utc>	Most recent accepted contribution
merge_conflicts	u32	Outstanding CRDT merge conflicts requiring human resolution
governance_model	GovernanceModel	Shared portfolio governance model
distribution_pending	Decimal	Amounts computed but not yet distributed to contributors

last_distribution_date	Option<DateTime<Utc>>	Date of last contributor distribution event
federation_nodes	Vec<NodeId>	Remote portfolio nodes participating in CRDT sync
health_score	ComputedColumn	PortfolioHealth including CollaborationScore component
resource_share_edges	Vec<GraphEdge>	Active ResourceShare edges from this portfolio

6. View Engine (KPVW)

The View Engine transforms raw PortfolioRow data into the shaped, filtered, sorted, grouped, and visually enriched spreadsheet that the user interacts with. Views are defined as ViewDefinition objects, serialized to JSON, versioned, and shareable — a view definition can be published to a portfolio template, shared with an organization, or embedded in an external page.

6.1 ViewDefinition Schema

Field	Description
id	UUID — view identifier
name	User-defined view name
sheet_id	Parent sheet this view belongs to (or 'master' for all-sheet views)
base_row_filter	Option<ViewFilter> — pre-filter applied before any user interaction (e.g., 'only rows owned by me')
column_schema	ColumnSchema — ordered list of visible columns with width, pin, and sort settings
filters	Vec<ViewFilter> — user-configurable filter predicates
sorts	Vec<ViewSort> — ordered sort predicates
groups	Vec<ViewGroup> — hierarchical grouping definitions
row_height	RowHeight: Compact Normal Tall Auto
highlight_rules	Vec<RowHighlightRule> — conditional color rules
pinned_columns	Vec<ColumnId> — columns always visible regardless of horizontal scroll
frozen_row_count	u32 — number of rows pinned at the top (e.g., for header summaries)
pivot_config	Option<PivotConfig> — pivot table mode configuration
chart_config	Option<ChartConfig> — chart overlay configuration
summary_row	Option<SummaryRowConfig> — aggregate row displayed at sheet bottom
board_config	Option<BoardConfig> — Kanban / Gantt / Calendar board mode settings
visibility	ViewVisibility: Private Shared Public Template
created_by	UserId
created_at / updated_at	Timestamps

6.2 Filter System

Filters are composable boolean predicates evaluated against row columns. Complex filter trees can be built by nesting AND and OR groups:

Filter Type	Syntax	Example
Equality	column = value	status = Active
Inequality	column != value	visibility != Private
Numeric Range	column BETWEEN a AND b	health_score BETWEEN 60 AND 100
Greater/Less	column > or < value	budget_utilization_pct > 80

Contains (string)	column CONTAINS substring	name CONTAINS 'API'
In Set	column IN [v1, v2, ...]	item_type IN [Project, Program]
Not In Set	column NOT IN [v1, ...]	status NOT IN [Archived, Cancelled]
Date Range	column AFTER / BEFORE / BETWEEN dates	due_date AFTER 2026-03-01
Is Empty	column IS EMPTY	due_date IS EMPTY
Is Not Empty	column IS NOT EMPTY	owners IS NOT EMPTY
Tag Includes	tags INCLUDES tag	tags INCLUDES 'q2-initiative'
Owner Is Me	owners INCLUDES me	owners INCLUDES me
Has Flag	risk_flags HAS_SEVERITY HIGH	risk_flags HAS_SEVERITY HIGH
Engine Score	health_score >= threshold	health_score >= 70
AND / OR Groups	(filter1 AND filter2) OR filter3	(status=Active AND health_score < 60) OR risk_score > 80

6.3 Sort, Group & Pivot

Feature	Configuration	Behavior
Sort	Vec<ViewSort { column, direction: Asc/Desc, null_placement: First/Last }>	Multi-column sort: primary sort on first entry; secondary on second; etc. Stable sort preserves relative order of equal rows.
Group	Vec<ViewGroup { column, collapse_default }>	Hierarchical grouping: first group becomes the outermost heading row; sub-groups nested beneath. Each group row shows aggregate values for its children.
Pivot	PivotConfig { row_field, col_field, value_field, aggregation: Sum/Avg/Count/Max/Min }	Cross-tabulation: rows become row_field values; columns become col_field values; cells show aggregated value_field. E.g., budget_spent by (owner × quarter).
Rollup	RollupConfig { hierarchy_column, rollup_columns }	Computes parent-row aggregate values from child rows in a hierarchy view. E.g., Portfolio row shows sum of all descendant budget_spent.
Summary Row	SummaryRowConfig { columns: Vec<(ColumnId, Aggregation)> }	Fixed aggregate row at bottom of sheet showing sum/avg/count/max/min of selected numeric columns.

6.4 Board Modes

Every sheet can be rendered in an alternative board mode. Board modes are view-layer transforms — the underlying PortfolioRow data is unchanged; only the rendering changes:

Board Mode	Configuration	Best For
Kanban Board	Columns = status values; cards show name, owner, due_date, health_score	Task and project status management; visual WIP limits
Gantt Chart	Rows = timeline components; horizontal bars = start_date to end_date/due_date; dependencies rendered as arrows	Project and program timeline planning
Calendar	Rows with date columns placed on a calendar grid by due_date or start_date	Scheduling, deadlines, milestone tracking

Agile Board	Sprint-scoped Kanban with story point totals per column	Sprint planning and execution
Resource Board	Rows = resources; columns = time periods; cells = allocation %	Capacity planning and conflict detection (AllocationEngine)
Network Graph	Nodes = rows; edges = GraphEdges; force-directed layout	Dependency visualization, portfolio relationship map
Treemap	Hierarchical tiles sized by a numeric column (e.g., budget_allocated)	Portfolio composition and budget distribution
Timeline Board	Swimlane rows = programs; bars = projects and milestones; nested structure	Cross-program roadmap view

7. Column System

7.1 Column Types

Column Type	Description	Storage
TextField	Free text string; sortable, searchable, filterable	String, indexed by SearchEngine
NumberField	Integer or decimal; supports aggregation (sum, avg, min, max)	Decimal, range-indexed
CurrencyField	Decimal with currency code; multi-currency aware; auto-conversion to base currency	Decimal + CurrencyCode
PercentField	Decimal 0–100; renders with % suffix; sortable and range-filterable	f32
DateField	Calendar date; supports range filters, relative filters ('next 7 days'), and calendar placement	Date, calendar-indexed
DateTimeField	Full timestamp; timezone-aware; rendered relative ('2 hours ago') or absolute	DateTime<Utc>
DurationField	Time span; rendered as '3d 4h'; supports arithmetic	Duration
EnumField	One-of a defined value set; renders as colored badge; fast equality filter	String enum, hash-indexed
MultiEnumField	Set of enum values; supports 'includes' filter	Vec<String>, set-indexed
RelationField	Foreign key reference to another PortfolioRow; renders the referenced row's name as a link	ComponentId, join-resolved
MultiRelationField	Set of foreign key references; renders as tag list of linked row names	Vec<ComponentId>
UserField	Reference to a user or organization entity; resolves to display name + avatar	EntityId
MultiUserField	Set of user/org references; renders as avatar cluster	Vec<EntityId>
TagField	Vec<String>; renders as tag chips; filterable by tag membership	Vec<String>, tag-indexed
BoolField	Boolean; renders as checkbox; filterable	bool
RichTextField	Markdown-formatted rich text; not sortable; searchable	Markdown String
FileRefField	Reference to a file or artifact stored in kogi-content	FileId, resolves to metadata + URL
URLField	Web URL; renders as clickable link	String
ComputedColumn	Derived value computed by Computation Engine from other columns; read-only	Computed at query time

AIColumn	Engine-generated signal: score, flag, recommendation, prediction; read-only	Written by kogi-engine via plugin writeback
FormulaColumn	User-defined formula over other columns (subset of ComputedColumn); power user feature	Evaluated by ColumnComputer
AuditColumn	Derived from EventLog; shows last actor, last action, change count	Derived from EventLog
AnalyticsColumn	Aggregated engagement or performance metric (view count, click count, engagement rate)	Updated by AnalyticsEngine

7.2 Custom Columns

Users and organizations can define custom columns on any sheet. Custom columns are typed, validated, and stored in the component's `ComponentData.properties` map (`Map<String, serde_json::Value>`). They participate fully in filtering, sorting, grouping, and export:

Custom Column Feature	Description
Add Column	Any user with Editor permission on the sheet can add a custom column of any supported type
Column Scope	Custom columns can be scoped to a single sheet (local) or promoted to all sheets (global) by an owner
Default Values	Custom columns support default values, required flags, and validation rules (min/max, regex, enum set)
Computed Custom Column	A FormulaColumn built from other columns using the formula editor (arithmetic, conditional, lookup)
Org Templates	Organizations can define standard column sets as ColumnTemplates, applied to all member portfolios
API Access	Custom column values are accessible via the kogi-dev API under <code>component.data.properties['column_key']</code>
Export	Custom columns appear in all export formats (CSV, XLSX, JSON) with user-defined column headers

8. Computation Engine (KPCM)

The Computation Engine powers all ComputedColumns, AIColumns, and rollup aggregations in the spreadsheet. It runs on two tiers: synchronous in-request computation for simple derived columns (arithmetic, rollup), and asynchronous kogi-engine computation for complex AI-driven signals (health score, risk score, match score, predictions). All computed results are cached in the CellStore with a staleness TTL and invalidated on relevant EventLog events.

8.1 Computational Models

Model	Inputs
PortfolioHealth	child component statuses, budget utilization, risk flags, progress %, active owner count, engagement
ProjectMetrics	sprint velocity history, backlog size, completion rate, cycle time, lead time
ProgramAlignment	child project health scores, KPI actuals vs targets, budget rollup, milestone status
SubPortfolioRollup	all descendant component financial and status data
ResourceUtilisation	resource allocation records, capacity units, consumed units, schedule
AssetValue	asset type, acquisition cost, valuation history, market data (if available)
ArtifactMaturity	artifact type, version count, review status, linked project completion
BinderCoverage	items in binder vs. required item types per template
BookConsistency	section completeness, required fields per book type, last updated
FolderOrganisation	folder depth, item count, orphan items, naming conventions
RecordIntegrity	required metadata fields, audit trail completeness, compliance tags
CollaborationScore	contributor count, contribution velocity, governance participation, conflict rate
BenefitsHealth	benefit account balances, coverage types, income replacement ratio, vesting progress

RiskScore	risk_flags (severity + probability), mitigation status, overdue items
IncomeProjection	gig earnings history, contract revenue, investment distributions, benefit contributions

8.2 ColumnComputer — Formula Engine

The ColumnComputer evaluates FormulaColumn definitions at query time. Formulas are defined in a type-safe expression language supporting:

- Arithmetic operators: +, −, ×, ÷, %, ^
- Comparison operators: =, !=, >, <, >=, <=
- Logical operators: AND, OR, NOT, IF(condition, true_val, false_val)
- String functions: CONCAT, UPPER, LOWER, LEN, CONTAINS, STARTSWITH
- Date functions: TODAY, DAYS_BETWEEN, DATE_ADD, QUARTER, YEAR, MONTH
- Aggregation functions: SUM, AVG, COUNT, MAX, MIN, MEDIAN (over a row's related children)
- Lookup functions: RELATED(relation_column, target_column) — dereference a RelationField and read a column from the related row
- Engine functions: HEALTH_SCORE, RISK_SCORE, MATCH_SCORE — inject engine-computed values as formula operands

8.3 Aggregation & Rollup

Group rows in the View Engine can display aggregate values for their member rows. Supported aggregations per column type:

Column Type	Aggregations	Notes
NumberField / CurrencyField / PercentField	SUM · AVG · MIN · MAX · MEDIAN · COUNT · COUNT_NON_EMPTY	SUM is default for financial columns; AVG for percentage/score columns
DateField / DateTimeField	MIN (earliest) · MAX (latest) · RANGE (latest – earliest)	Useful for 'earliest deadline in group' or 'project span'
EnumField	COUNT_BY_VALUE (distribution) · MODE (most common value)	Rendered as mini bar chart in group row
TagField	UNION (all tags) · INTERSECTION (shared tags) · COUNT_BY_TAG	Displayed as tag cloud in group header
RelationField	COUNT · COUNT_UNIQUE	Count of related rows
UserField	COUNT_UNIQUE · LIST	Count of distinct users; list as avatar cluster
BoolField	COUNT_TRUE · COUNT_FALSE · PCT_TRUE	E.g., '7 / 12 tasks complete (58%)'
ComputedColumn / AIColumn	AVG · MIN · MAX · MEDIAN	Aggregate the computed signal across group members

9. Collaborative Editing (KPCL)

The Portfolio Spreadsheet is a live, multi-user, collaboratively editable document. Multiple users — across multiple devices, time zones, and nodes — can simultaneously view and edit the same portfolio. Consistency is maintained by the CRDT engine (LWW field writes + OR-Set for set-valued columns), and every mutation is appended to the immutable EventLog.

9.1 CRDT Operations

CRDT Operation	Description
SetField (LWW)	Last-Write-Wins field update: write a scalar or enum column value with a VectorClock timestamp. If two concurrent writes target the same cell, the one with the higher VectorClock timestamp wins.
AddToSet (OR-Set)	Add a value to a set-valued column (tags, owners, policy_ids, toolbox_ids) with a unique tag. Concurrent adds both survive; no value is lost.
RemoveFromSet (OR-Set)	Remove a specific tagged entry from a set-valued column. Removes only the explicitly tagged entry; concurrent adds of the same value survive (OR-Set semantics).
AppendLog	Append an EventLog entry. Append is commutative — all concurrent appends survive; ordering is by VectorClock causal timestamp.
AttachChild / DetachChild	Add or remove a GraphEdge::Hierarchy edge. Concurrent attach operations both survive; concurrent detach of different edges both succeed; concurrent attach + detach of the same edge resolves to the higher-timestamp operation.
StatusTransition	Status column mutations are not LWW — they follow the lifecycle state machine. Invalid transitions are rejected by the PolicyEngine even if the VectorClock wins.

9.2 Conflict Resolution

Conflict Type	Resolution Strategy
Concurrent scalar field writes (LWW)	Higher VectorClock timestamp wins. Losing write is preserved in EventLog as a ConflictRecord for human review.
Concurrent set additions	Both values survive (OR-Set). No conflict — both are correct.
Concurrent add and remove of same set value	Remove wins if the remove's VectorClock dominates; otherwise add wins. Displayed to user as a ConflictFlag on the row.
Concurrent status transitions to incompatible states	Both are rejected; the component status reverts to the last confirmed state. A MergeConflict event is appended; users are notified via KNTF alert.
Concurrent structural edits (attach/detach child)	Higher-timestamp operation wins. Conflicting edit preserved in EventLog. Oba notifies both actors of the resolution.
Concurrent contributions to shared portfolio	Handled by CollaborationEngine: both contributions submitted to steward review queue. Steward merges manually or approves both independently.
CRDT sync failure across federated nodes	Operations buffered in CrdtLog until connectivity restored. Sync replays buffered operations in causal order on reconnect.

9.3 Contribution Attribution in Collaborative Sheets

For shared portfolio components and crowdresourced sheets, every row mutation generates a ContributionRecord attached to the component's EventLog. The CollaborationEngine continuously recomputes attribution weights:

```
ContributionRecord {
```

```
record_id:          UUID,
contributor_id:     EntityId,          // user | org | collective
contribution_type:  ContributionType,  // Labor | Capital | Asset | Knowledge | Artifact
| Code | Design | Data
portfolio_component_id: ComponentId,
contribution_value: ContributionValue { amount: Decimal, unit: String },
attribution_weight: f64,                // proportional weight in distribution
calculations
governance_status: GovernanceStatus,   // PendingReview | Accepted | Rejected | Merged
linked_ledger_entry: Option<LedgerId>,  // kogi-bank entry for capital contributions
}
```

Attribution Weight Formula	Description
Labor-based	weight = hours_contributed / total_hours_contributed_by_all
Equity-based	weight = equity_share_pct / 100
Hybrid (Cooperative Default)	weight = 0.60 × labor_weight + 0.30 × equity_weight + 0.10 × governance_participation_weight
Custom	Organization defines formula via GovernanceProposal; CollaborationEngine evaluates against ContributionRecords

10. Portfolio Health Model — Full Specification

The PortfolioHealth model is the master analytical output of the Computation Engine for Portfolio and SubPortfolio rows. It computes an overall health_score (0–100) from a weighted composition of seven sub-dimensions. The health score is the primary AI-derived column on every Portfolio and Program row — it is the single most important computed signal for portfolio management decisions.

10.1 Health Score Dimensions

Dimension	Default Weight	Inputs	Score Logic
Delivery Health	25%	progress_pct, schedule_variance_days, milestone completion rate	Full score if on-track; decreasing score for schedule variance; zero for abandoned milestones
Financial Health	20%	budget_utilization_pct, cash_flow, ROI trend, overrun_flags	Full score if utilization < 80%; decreasing score for overruns; penalty for flagged budget anomalies
Risk Health	20%	risk_score, open_risk_count, critical_risk_count, mitigation_completion_rate	Inverse of risk_score; full score if no open critical risks; decays rapidly with unmitigated high-severity risks
Resource Health	15%	resource_utilization_pct, over_allocation_flags, toolbox coverage	Full score if utilization 60–80%; penalty for over-allocation (>100%) and under-allocation (<30%)
Engagement Health	10%	engagement_rate, follower_growth, active_contributor_count, collaboration_score	Measures portfolio visibility and community engagement; relevant for public and shared portfolios
Governance Health	5%	open_approval_requests, compliance_flags, overdue_reviews, charter_present	Full score if all approvals resolved; decreasing score for unresolved compliance flags and overdue reviews
Benefit Coverage Health	5%	benefits_health_score, coverage_gap_flags, pto_remaining_days	Relevant for worker's personal portfolio; scores adequacy of portable benefits coverage

10.2 Health Score Computation

```

fn compute_portfolio_health(portfolio_id) -> PortfolioHealth {
  let children      = fetch_all_descendants(portfolio_id);
  let delivery_h    = score_delivery(children);    // schedule variance, milestone completion
  let financial_h   = score_financial(portfolio_id, children); // budget, ROI
  let risk_h        = score_risk(children);        // risk_flags aggregation
  let resource_h    = score_resources(children);   // allocation utilisation
  let engagement_h  = score_engagement(portfolio_id); // analytics
  let governance_h  = score_governance(children);  // approvals, compliance
  let benefits_h    = score_benefits(owner_id);    // benefit account coverage

  let health_score =
    delivery_h * 0.25 + financial_h * 0.20 + risk_h * 0.20
    + resource_h * 0.15 + engagement_h * 0.10 + governance_h * 0.05
    + benefits_h * 0.05;

```

```
PortfolioHealth { health_score, dimensions: [...], anomalies: [...] }  
}
```


11. kogi-engine Integrations

The Portfolio Spreadsheet is both a data source and a display target for the full kogi-engine intelligence stack. Every engine contributes computed columns, derived rows, anomaly flags, and Oba recommendations to the spreadsheet. The integration protocol is bidirectional: the spreadsheet emits EventLog events to the engine; the engine writes computed values back via the plugin writeback protocol.

Engine	Columns Contributed to Spreadsheet
AnalyticsEngine	views, clicks, ctr, engagement_rate, followers, spread, share_count, sentiment_score
TelemetryEngine	flow_stats, event_count, last_event_timestamp, mutation_velocity
RecommendationEngine	recommendation_score, similar_items, suggested_actions
PersonalizationEngine	personalized_rank, relevance_to_active_persona, feed_visibility_score
MatchEngine	match_score (for opportunity queries), talent_match_score (for resource rows), investor_match_score (for campaign rows)
GraphEngine	dependency_depth, critical_path_flag, network centrality, cluster_membership
RiskEngine	risk_score, risk_breakdown, mitigation_gap_flags, at_risk_deadline_count
OptimizationEngine	resource_optimization_hints, budget_reallocation_suggestions, schedule_optimization_hints
SearchEngine	search_rank, full_text_index_status, keyword_suggestions
AllocationEngine	capacity_conflicts, allocation_recommendations, utilization_heatmap
IncentiveEngine	kogipoints_earned, incentive_streak, contribution_badges
GameEngine	level, xp_points, badge_refs, community_rank
BenefitsEngine	benefits_health_score, coverage_gap_flags, tax_savings_estimate, contribution_forecast
GrantEngine	grant_match_score, eligibility_flags, recommended_grants
CrowdfundingEngine	campaign_health_score, success_probability, investor_match_quality
CollaborationEngine	collaboration_score, contributor_diversity_index, merge_conflict_count
BookingEngine	availability_conflicts, booking_revenue_forecast, utilization_calendar
CRMEngine	lead_score, pipeline_value, follow_up_due_flags
LogisticsEngine	logistics_status, itinerary_health, resource_routing_conflicts

12. Structural Primitives — Group, Collection, List, Schedule, Directory

Structural primitives are lightweight, in-memory organizing constructs that sit above individual rows but below Container components. They are used internally by the View Engine (for grouping) and by the Portfolio System's query layer (for structuring result sets). They also appear as first-class columns on certain sheet types.

12.1 Primitive Types

Primitive	Definition	Use in Spreadsheet
Group	A labeled cluster of rows linked by a shared property value (e.g., all rows where status=Active). Dynamic — membership changes as column values change.	Group rows in board/list view; used by ViewGroup in KPVW; summary row aggregates group members
Collection	An unordered, persistent, user-defined set of component IDs. Static — membership changes only by explicit add/remove. Equivalent to a custom filter saved as a named set.	Saved row sets; 'pinned' item collections; quick-access views; shared curated lists
List	An ordered, persistent sequence of component IDs. Static with explicit ordering. Supports drag-and-drop reorder.	Prioritized backlogs; ranked lists; ordered workflows; reading lists; TODO sequences
Schedule	A causal list of items where ordering is determined by start_date and dependencies (dependency graph sort). Computable from project tasks.	Sprint boards; project schedules; gantt row ordering; timeline rendering
Directory	A spatial collection of items organized by hierarchical category paths (folder-like). Items are addressed by path: /programs/q2/projects/api-redesign.	Filesystem-like navigation of large portfolios; org directory of members; asset catalogue by category

12.2 Container Types — Detailed

Container	Subtype	Primary Content
Binder	General	Any items organized by logic: client, topic, campaign
Book	Notebook	Freeform notes and documents as pages
Book	ContactBook	Contact profiles linked from KCON linkforest
Book	PlayBook	Repeatable workflow templates and process docs
Book	ScheduleBook	Schedules, calendar items, and booking blocks
Book	PlanBook	Strategic plans, roadmaps, and OKR documents

Book	GuideBook	Documentation sets, SOPs, and frameworks
Book	ItemBook	Structured catalogs of items with full metadata
Book	ItemBook:Charter	Governance charters and board documents
Book	ItemBook:Catalogue	Product, service, or asset catalogs
Book	ItemBook:Library	Knowledge and resource libraries
Book	ItemBook:Template	Reusable component templates for fast creation
Book	ItemBook:Log	Operational logs, incident records, audit trails
Record	Formal	Auditable formal records: contracts, agreements, submissions
Folder	General	Hierarchical file-system-like item organizer
Registry	Asset	Structured searchable catalog of assets with types
Registry	Skills	Searchable skills registry for a user or org
Registry	Vendor	Third-party vendor, supplier, or platform registry
Archive	Deep Storage	Completed/retired items with full restore capability

13. API Surface (KPMS · REST + gRPC)

13.1 Portfolio REST API (External)

Endpoint	Method	Description
GET /portfolio/:id	GET	Read a single PortfolioComponent by ID (visibility-gated)
GET /portfolio/@:slug	GET	Read component by slug
POST /portfolio	POST	Create a new component (item or container)
PATCH /portfolio/:id	PATCH	Update component fields (owner/editor only)
DELETE /portfolio/:id	DELETE	Soft-archive component
GET /portfolio/:id/children	GET	List direct children in hierarchy
GET /portfolio/:id/descendants	GET	List all transitive descendants
POST /portfolio/:id/attach/:child_id	POST	Attach a child component (creates Hierarchy edge)
DELETE /portfolio/:id/attach/:child_id	DELETE	Detach a child component
POST /portfolio/:id/actions	POST	Perform an ActionKind on a component (like, share, follow, invest, donate, post, tag)
GET /sheets	GET	List all sheets accessible to the authenticated user
GET /sheets/:sheet_id	GET	Get sheet definition and metadata
GET /sheets/:sheet_id/rows	GET	Query rows on a sheet (with filter, sort, group, pagination)
POST /sheets	POST	Create a custom sheet
PATCH /sheets/:sheet_id	PATCH	Update sheet definition
GET /views	GET	List saved views
POST /views	POST	Create a saved view
PATCH /views/:view_id	PATCH	Update a saved view
GET /views/:view_id/rows	GET	Render rows for a specific saved view
GET /portfolio/:id/analytics	GET	Get computed analytics for a component
GET /portfolio/:id/health	GET	Get PortfolioHealth model output
GET /portfolio/:id/eventlog	GET	Get EventLog for a component
POST /portfolio/:id/snapshot	POST	Create a snapshot
POST /portfolio/restore/:snapshot_id	POST	Restore from a snapshot
GET /portfolio/:id/allocations	GET	Get resource allocations for a component
POST /portfolio/:id/allocate	POST	Create a resource or budget allocation
POST /portfolio/:id/record-spend	POST	Record budget spend
GET /portfolio/search	GET	Full-text + faceted search across the registry
GET /portfolio/query	GET	PQL (Portfolio Query Language) query endpoint
GET /portfolio/:id/contributions	GET	List ContributionRecords for a shared component

POST /portfolio/:id/contribute	POST	Submit a contribution to a shared component
POST /portfolio/:id/crdt/sync	POST	Submit a CrdtLog batch for synchronization
GET /portfolio/export/:sheet_id	GET	Export sheet as CSV, XLSX, or JSON (content-type negotiation)

13.2 Portfolio Query Language (PQL)

PQL is a structured query interface for the portfolio registry — SQL-inspired but typed to the PortfolioRow schema. It is exposed via the /portfolio/query REST endpoint and the QueryService gRPC method.

```
-- Example PQL queries
-- All active projects with health score below 70, sorted by due date:
SELECT id, name, status, health_score, due_date, owners
FROM rows
WHERE item_type = 'Project'
      AND status = 'Active'
      AND health_score < 70
ORDER BY due_date ASC NULLS LAST;

-- Q2 budget rollup by program:
SELECT program_ref.name, SUM(budget_allocated), SUM(budget_spent),
      SUM(budget_remaining), AVG(budget_utilization_pct)
FROM rows
WHERE item_type IN ('Project', 'Program')
      AND quarter = 'Q2-2026'
GROUP BY program_ref.name
ORDER BY SUM(budget_spent) DESC;

-- Benefit accounts with coverage gaps:
SELECT id, name, benefit_type, balance, coverage_gap_flag, tax_savings_estimate
FROM rows
WHERE item_type = 'BenefitAccount'
      AND coverage_gap_flag IS NOT EMPTY
ORDER BY benefit_type;

-- Grant matching: top 10 grants by portfolio match score:
SELECT id, name, source, target_amount, deadline, portfolio_match_score
FROM rows
WHERE item_type = 'Grant'
      AND status IN ('Researching', 'Drafting')
ORDER BY portfolio_match_score DESC
LIMIT 10;
```

13.3 Internal gRPC Services

Service	Methods
PortfolioService	CreateComponent · ReadComponent · UpdateComponent · ArchiveComponent · AttachChild · DetachChild · AddDependency · SearchComponents · QueryPQL
SheetService	ListSheets · GetSheet · GetSheetRows · CreateSheet · UpdateSheet · DeleteSheet · ExportSheet
ViewService	ListViews · GetView · GetViewRows · CreateView · UpdateView · DeleteView · ShareView
ColumnService	ListColumns · GetColumnSchema · AddCustomColumn · UpdateColumn · RemoveColumn · GetComputedValue

AnalyticsService	GetComponentAnalytics · GetPortfolioHealth · GetProjectMetrics · GetProgramAlignment · GetRiskScore · GetIncomeProjection
AllocationService	GetAllocations · AllocateBudget · AllocateResource · RecordSpend · RecordConsumption · GetOverruns
GovernanceService	AttachPolicy · DetachPolicy · RequestApproval · ResolveApproval · GetComplianceFlags · GetRiskFlags
CRDTSERVICE	ApplyCrdtBatch · GetCrdtLog · SyncFederation · GetConflicts · ResolveConflict
SnapshotService	TakeSnapshot · ListSnapshots · RestoreSnapshot · GetCheckpoints
ContributionService	ListContributions · SubmitContribution · ReviewContribution · GetAttributionWeights · TriggerDistribution
ExportService	ExportCSV · ExportXLSX · ExportJSON · ExportJSONLD · GetExportStatus
PluginService	RegisterPlugin · UnregisterPlugin · ListPlugins · AttachToolbox · DetachToolbox · GetToolboxIds

14. Persistence & Storage Architecture

14.1 Storage Layer Design

Store	Technology	What Is Stored
Primary Component Store	PostgreSQL + JSONB	PortfolioRow canonical records — ComponentMetadata + ComponentData serialized as JSONB with typed indexed columns for hot query fields (status, owner, type, created_at, health_score)
Graph Store	PostgreSQL (adjacency table)	GraphEdge table: source_id · target_id · edge_type · metadata JSONB · created_at. Indexed on source, target, and type for O(1) adjacency lookups
EventLog Store	PostgreSQL (append-only, partitioned by created_at)	Immutable event records: component_id · actor_id · event_type · delta JSONB · vector_clock · timestamp. Partitioned by month for efficient historical queries
CRDT Buffer	Redis (hash by component_id)	CrdtLog operations awaiting apply; vector clock cache; hot-path LWW cell cache for sub-millisecond concurrent writes
Cell Cache	Redis (hash by component_id:column_id)	Computed column values cached with TTL; invalidated on EventLog trigger. Prevents re-computation on every read for expensive AI columns
Search Index	Meilisearch (or Typesense)	Full-text index over component name, tags, bio, description, custom fields. Rebuilt on component create/update via plugin hook
Snapshot Store	S3-compatible object store	Full PortfolioSystem snapshots serialized as compressed JSON. Keyed by snapshot_id; lifecycle-managed with configurable retention
Analytics Store	ClickHouse (columnar)	High-throughput append of analytics events: views, clicks, engagements, shares. Queried by AnalyticsEngine for aggregation; never modified after write
Engine Feature Store	Redis + PostgreSQL (materialized views)	Pre-computed engine signals (health_score, risk_score, match_score) stored as materialized views, refreshed on schedule and on trigger

14.2 Indexing Strategy

Index	Type
Primary Key Index	B-Tree (PostgreSQL)
Owner Index	B-Tree
Type Index	B-Tree
Status Index	B-Tree
Visibility Index	B-Tree
Timestamp Indexes	B-Tree
Financial Indexes	B-Tree
Score Indexes	B-Tree
Tag Index	GIN (PostgreSQL)
Full-Text Index	Meilisearch

Hierarchy Edge Index	B-Tree (GraphEdge table)
EventLog Partition Index	B-Tree (partitioned by month)

14.3 Snapshotting & Recovery

Operation	Trigger	Storage
Auto Snapshot	Daily at 02:00 UTC per active portfolio	S3 — named {portfolio_id}/{date}.snap.json.gz
Manual Snapshot	User or Oba triggers via UI/API	S3 — named {portfolio_id}/{user_id}/{timestamp}.snap.json.gz
Checkpoint	Before any bulk operation (bulk status change, import, federation sync)	Redis (temporary) + S3 (permanent after confirmation)
Restore	User selects snapshot from Snapshot Sheet; confirm dialog required	PortfolioSystem::restore_snapshot() replays all ComponentData from snapshot; EventLog entries from snapshot to now preserved as post-restore history
Archive	User archives a component or portfolio	Component status set to Archived; deep-stored in Archive container; removable from active sheet views but queryable via SHT-028
Federation Sync	CRDT sync between federated portfolio nodes (PortfolioFederation::sync_crdt)	CrdtLog operations exchanged; VectorClock merged; conflicts flagged for human resolution

15. Security, Access Control & Compliance

15.1 Permission Tier Hierarchy

Tier	Level
Owner	5
Admin	4
Editor	3
Contributor	2
Viewer	1
Public (Unauthenticated)	0

15.2 Row-Level & Column-Level Access Control

Mechanism	Description
Row-Level Security (RLS)	Every query is automatically scoped to rows where the caller satisfies the PermissionTier for that row's visibility level. Implemented as a PostgreSQL RLS policy on the component table.
Column-Level Visibility	Individual columns can be marked Private (owner-only), Trusted (shared with connections), or Public. The API masks column values based on the caller's PermissionTier.
PolicyEngine	Every mutation passes through the PolicyEngine trait — custom governance rules (e.g., 'all budget allocations > \$10k require GovernanceVote') are evaluated before the write is accepted.
Shared Portfolio ACL	Shared portfolios carry a detailed ACL: owner_ids, admin_ids, editor_ids, contributor_ids, viewer_ids — all checked on every mutation.
EventLog Immutability	EventLog entries are append-only and cryptographically chained (hash of previous entry included in each new entry). No EventLog entry can be deleted — only archived to cold storage.
PII Protection	Legal name, phone numbers, private emails, and financial account details are classified as PII; stored with field-level encryption; never returned in bulk export without explicit consent.
Audit Trail	All read accesses to sensitive fields (legal name, financial data, benefit account balances) are logged as AuditLog entries separate from the main EventLog.
Data Portability	Users can export their full portfolio data (GDPR/CCPA compliance): all components, event logs, analytics, contributions, and financial summaries as a JSON-LD or CSV archive.

16. Export, Integration & Embedding (KPEX)

16.1 Export Formats

Format	API / Method	Notes
CSV	GET /portfolio/export/:sheet_id?format=csv	Standard delimiter-separated export of visible columns. Computed columns included as static values at export time.
XLSX	GET /portfolio/export/:sheet_id?format=xlsx	Multi-sheet Excel workbook: main sheet + summary pivot + chart data. Uses openpyxl generation pipeline.
JSON	GET /portfolio/export/:sheet_id?format=json	Raw PortfolioRow JSON array — full schema including all fields, relationships, and computed values.
JSON-LD	GET /portfolio/export/:sheet_id?format=jsonld	Linked Data export with schema.org and kogi-vocab type annotations — portable, semantic, and machine-readable.
Markdown	GET /portfolio/export/:sheet_id?format=md	Markdown document: section headings per group, tables per component type. Suitable for portfolio showcases and README generation.
PDF Report	GET /portfolio/export/:sheet_id?format=pdf	Rendered portfolio report document — styled with kogi brand template, includes charts, health summaries, and key metrics.
vCard Bundle	GET /portfolio/export/contacts?format=vcf	Exports ContactBook rows as a vCard (.vcf) bundle for import into external contact managers.
Webhook Push	POST /portfolio/webhooks	Real-time push of EventLog events or row change deltas to a registered external URL on every mutation.
Embed Widget	GET /portfolio/embed/:view_id	Returns an embeddable iframe/script tag that renders a read-only view of a public sheet or portfolio in an external webpage.
Developer API	GET /portfolio/:id (REST) + PortfolioService gRPC	Full programmatic access via kogi-dev SDK.

16.2 Import Sources

Import Source	Import Method
CSV / XLSX Upload	File upload → column mapping wizard → batch create
Google Sheets	OAuth + Sheets API → pull → import wizard
Notion	Notion integration (API key) → database pull
Airtable	Airtable API → base pull
Jira	Jira API → issue pull
Asana	Asana API → project pull
GitHub / GitLab Issues	Issues API → pull
Quickbooks / Wave	Accounting API → transaction pull

Gig Platform Earnings	Platform API or CSV upload
ContactBook Import	vCard, Google Contacts, Outlook CSV

17. Portfolio Template System

Templates are pre-configured PortfolioSystem instances — pre-populated with sheets, column schemas, views, computed models, row scaffolding, and Oba automation rules. Any portfolio can be saved as a template and instantiated as many times as needed. Templates are distributed via the KOGI-APPSTORE.

17.1 Template Types

Template	Pre-configured For	Included Sheets & Views
Independent Worker Starter	Solo freelancer / gig worker; first portfolio setup	Master Registry, Work & Gigs, Finances, Portable Benefits, Timeline, Contacts
Software Developer Portfolio	Developer with projects, open-source contributions, and client contracts	Projects (Agile board default), Tasks & Backlog, Assets (repositories), Artifacts (releases), Finances, Contacts
Creative Portfolio	Designer, artist, musician, content creator	Projects, Artifacts (creative works), Marketplace Listings, Finances, Community Feed, Contacts
Freelance Consultant	Consultant with multiple client engagements	Work & Gigs (contract-focused), CRM view, Finances, Timeline, Contracts & Deliverables, Contacts
Cooperative / Collective	Multi-member cooperative or collective	Shared Portfolios, Group Economics, Collaboration, Governance, Finances (cooperative treasury), Organizations
Startup Founder	Early-stage startup with team, product, investors	Projects, Team (shared), Equity & Crowdfunding, Grants, Roadmap, Finances, Contacts, Risk Register
Gig Worker Benefits Optimizer	Gig worker focused on portable benefits and income optimization	Work & Gigs, Portable Benefits, Finances, Grant & Microfinancing, Income Projection view
Research & Academic	Researcher, academic, or knowledge worker	Projects (research methodology), Artifacts (papers, datasets), Grants, Timeline, Contacts, Knowledge Registry
Event Producer / Booking	Event or touring professional with bookings, crew, logistics	Work & Gigs (booking-focused), Timeline (tour calendar), Resources (crew, equipment), Finances, Logistics view
Organization / DAO	Formal organization with governance, treasury, and member management	Organizations, Shared Portfolios, Group Economics, Governance, Finances (treasury), Member Directory

18. Oba AI Integration — Spreadsheet Intelligence

Oba is the AI chief-of-staff that lives inside the portfolio spreadsheet. It reads every row, every column, and every engine signal — and surfaces proactive, context-aware intelligence directly in the spreadsheet UI. Oba does not just answer questions; it notices what the user should notice, flags what they should act on, and prepares actions for their approval.

18.1 Oba Capabilities in the Spreadsheet

Capability	Description
Row Annotations	Oba attaches inline annotations to rows: 'This project is 14 days behind schedule — adjust due date or reduce scope?' · 'This grant deadline is in 5 days — draft application now?'
Column Completion	Oba suggests values for empty required fields: 'Due date is missing — suggest April 30 based on similar projects?' · 'No risk flags — would you like me to analyze and create an initial risk assessment?'
Smart Filters	'Show me everything at risk this week' → Oba translates natural language to ViewFilter: { risk_score > 70 OR due_date < TODAY+7 AND status = Active }
Formula Suggestions	When a user adds a custom column, Oba suggests formulas: 'You're tracking revenue and expenses — want me to add a profit margin % formula column?'
Batch Actions	'Archive all completed projects older than 90 days' → Oba prepares a bulk archive action for user confirmation before executing
Sheet Generation	'Create a sheet showing all my income sources this quarter grouped by platform' → Oba generates a ViewDefinition and creates the sheet
Health Briefing	Daily Oba portfolio briefing: top health risks, upcoming deadlines, pending approvals, contribution opportunities, suggested grant matches
Anomaly Alerts	Oba proactively flags: budget overrun approaching (>80% utilization), velocity drop in a sprint, stalled project (no updates in 14 days), coverage gap in benefits
Import Mapping	During CSV/XLSX import, Oba suggests column mappings: 'This looks like 'Project Name' — map to PortfolioRow.name?'
Competitor Benchmarking	For public portfolio rows, Oba compares health and performance against anonymized platform benchmarks: 'Your project velocity is in the top 30% of similar projects'
Contribution Matching	For crowdresourced components, Oba matches potential contributors from the user's network: 'Alex has the skills listed in this project's requirements — send collaboration invite?'
Narrative Generation	'Write a portfolio highlight for my top 3 projects this quarter' → Oba generates a structured portfolio narrative from row data — for resumes, proposals, and grant applications

19. Platform Integration Map

kogi-* Module	Dependency on Portfolio Spreadsheet	Data Flow Direction
kogi-home	Reads Master Registry for dashboard overview: active program count, portfolio health summary, wallet balance, upcoming deadlines	Read: portfolio → home dashboard
kogi-office (boards/timelines)	Reads SHT-004 (Projects) + SHT-005 (Tasks) for board and gantt rendering; writes task status transitions and sprint completions back	Read + Write: portfolio ↔ office
kogi-bank	Reads SHT-009 (Finances), SHT-015 (Benefits), SHT-016 (Grants), SHT-017 (Crowdfunding); writes payment events, benefit balance updates, and ledger transactions back	Read + Write: portfolio ↔ bank
kogi-marketplace	Reads SHT-026 (Marketplace Listings) for published assets and resources; writes investment/donation action events; reads talent discovery from SHT-006 (Resources)	Read + Write: portfolio ↔ marketplace
kogi-community	Reads public rows from Master Registry for community feed; writes engagement events (likes, shares, mentions) back to analytics columns	Read + Write: portfolio ↔ community
kogi-exchange	Reads SHT-017 (Exchange), SHT-027 (Deals); writes deal execution events and matched allocation records	Read + Write: portfolio ↔ exchange
kogi-profile	Profile rows are PortfolioComponents in SHT-025 (Contacts); linktree leaves are stored as Resource rows; linked accounts sync to asset columns	Read + Write: portfolio ↔ profile
kogi-engine	Reads all sheets via EventLog subscription for feature extraction; writes computed column values back via plugin writeback protocol	Read + Write: portfolio ↔ engine
kogi-studio	Reads and writes Artifact rows (creative outputs); links design assets to parent Project rows via Hierarchy edges	Read + Write: portfolio ↔ studio
kogi-dev (API/SDK)	Full read/write access to all sheets and rows via kogi-dev API; PortfolioPlugin trait for custom computed columns and automation rules	Full bidirectional: portfolio ↔ developer
KOGI-MANAGER	Reads PolicyEngine attachments; writes RBAC policies and governance configurations; enforces PermissionTier hierarchy on all mutations	Policy: KOGI-MANAGER → portfolio
KOGI-APPSTORE	Distributes Portfolio Templates, custom column sets, computed model plugins, and view theme packages	Distribution: APPSTORE → portfolio templates

20. Open Items & Development Roadmap

20.1 Current Open Items (v2.2 → v3.0)

Item	Priority	Description
Persistence Layer Pluggability	P0	PortfolioSystem is currently in-memory with PostgreSQL being integrated. Full PortfolioStore trait abstraction needed for pluggable backends (SQLite for local, PostgreSQL for cloud, CouchDB for peer-to-peer).
Status CRDT Merge Strategy	P0	apply_crdt_op() currently skips status mutations. A custom CRDT lattice based on ComponentStatus lifecycle ordering must be specified before multi-node deployments.
EventLog Flush to Data Lake	P1	10,000 event cap with FIFO eviction risks historical loss. Plugin hook must flush older events to ClickHouse / S3 before eviction, with full restoration path.
TMS Deep Integration	P1	v2.1 introduces opaque ToolBoxId references. Full ToolBox lifecycle event propagation (creation, deletion, version bumps) back to referencing components must be specified.
AI Agent Writeback Protocol	P1	A typed, permission-respecting protocol for kogi-engine to write computed values back to portfolio rows — creating Metric entries, appending governance_notes, updating compliance_flags.
Formula Engine (v1)	P1	ColumnComputer formula engine is defined but not fully implemented. v3.0 must ship the complete expression language evaluator with all listed functions.
Real-Time Collaborative UI	P2	WebSocket-based live update propagation from CRDT mutations to all viewing clients — the 'live cursor' multi-user spreadsheet experience.
PQL Full Implementation	P2	PQL parser and evaluator must be implemented against the PostgreSQL storage layer with full JOIN support for relation columns.
Spreadsheet Import Pipeline	P2	CSV/XLSX import with Oba-assisted column mapping wizard and batch row creation with progress tracking.
Template Marketplace	P3	KOGI-APPSTORE integration for distributing, rating, and purchasing portfolio templates; template versioning and update notifications.
On-Chain Mechanism Execution	P3	For high-value cooperative distributions and governance votes: integration with on-chain execution layer for tamper-proof finalization.
Federated MatchEngine	P3	MatchEngine operating across ShangoOS federated portfolio nodes — cross-federation talent and resource matching.

20.2 Version Roadmap

Version	Milestone	Key Deliverables
v2.2 (Current)	Portfolio Spreadsheet Foundation	Master Registry Sheet · Hierarchy Sheet · Projects Sheet · Finances Sheet · Benefits Sheet · Work & Gigs Sheet · PortfolioHealth v1 · Basic CRDT · PostgreSQL persistence
v2.5	Intelligence & Collaboration	AI ComputedColumns (health, risk, income projection) · CollaborationEngine · Shared Portfolios · Contribution Attribution · CrowdfundingCampaign · Grant & Crowdfunding Sheets
v3.0	Full Spreadsheet Platform	PQL · Formula Engine · All 30 Sheets · All Board Modes · Template Marketplace · Real-time Collaboration UI · Full Import/Export Pipeline · Oba Spreadsheet Intelligence

v3.5	Federation & Ecosystem	Federated portfolio sync · Cross-org MatchEngine · On-chain governance execution · Third-party sheet integrations (Google Sheets, Airtable, Notion) · KOGI-APPSTORE plugin ecosystem
v4.0	Autonomous Intelligence	Oba autonomous portfolio management (approval-first) · Predictive analytics and scenario simulation · AI-generated portfolio narratives · Cross-platform reputation portability via verifiable credentials